

UNIVERSIDAD NACIONAL DE SAN ANTONIO
ABAD DEL CUSCO
FACULTAD DE INGENIERÍA ELÉCTRICA, ELECTRÓNICA,
INFORMÁTICA Y MECÁNICA
ESCUELA PROFESIONAL DE INGENIERÍA INFORMÁTICA Y DE
SISTEMAS



INFORME TÉCNICO DE DESARROLLO DE LA INTERFAZ DE USUARIO (UI/UX)

Sesión 06: Modelo de Restricciones y Prevención de Errores de Layout

Constraints go down, sizes go up, parent sets position

Asignatura: Desarrollo de Software II (IF616AIN / IF616BIN) — Semestre 2026-II

Unidad: I - Fundamentos de Dart, UI declarativa y UX

Proyecto: Portafolio Profesional y Tablero de Restricciones del Estudiante

Docente: Mtro. Ing. Yover Collantes Valer

Integrantes / Estudiante: Saire Bustamante, Edmil Jampier (Código: 174449)

Fecha de Entrega: Cusco, 7 de Octubre de 2026

URL GitHub: <https://github.com/milith0kun/desarrollo-de-software-2>

1. Introducción y Objetivos

1.1. Descripción del Proyecto

El presente proyecto corresponde al desarrollo de una aplicación móvil integral en Flutter titulada «**Portafolio Profesional y Tablero de Restricciones del Estudiante**». Esta solución tecnológica está diseñada para centralizar la información académica, habilidades técnicas, proyectos de ingeniería y métricas de desempeño universitario del estudiante de la Escuela Profesional de Ingeniería Informática y de Sistemas de la UNSAAC, ofreciendo además un módulo interactivo para la visualización, simulación y diagnóstico en tiempo real de los comportamientos del motor de renderizado de Flutter.

1.2. Objetivo de la Interfaz

El propósito primordial de la interfaz de usuario (UI/UX) es:

- Proporcionar una experiencia visual altamente profesional, accesible y responsiva, inspirada en los colores y lineamientos de identidad visual institucional de la UNSAAC (Granate #7A1F2B, Dorado #C9971F y Azul pizarra #1E293B).
- Demostrar el dominio riguroso del **modelo de restricciones** de Flutter, garantizando que cada componente visual se adapte dinámicamente a distintas densidades de pantalla y orientaciones sin presentar desbordamientos (`RenderFlex overflowed`) ni bloqueos por restricciones infinitas (`unbounded constraints`).
- Proveer una arquitectura modular que permita navegar con fluidez entre el perfil profesional del estudiante, los laboratorios guiados y el banco de autoevaluación teórico-práctica.

1.3. Alcance de la Sesión

Conforme a la programación curricular de la Primera Unidad Didáctica (*Fundamentos de Dart, UI declarativa y experiencia de usuario*):

Delimitación del Alcance Técnico

Esta entrega comprende **exclusivamente la construcción de la interfaz de usuario estática** empleando widgets inmutables de tipo `StatelessWidget`. Queda fuera del alcance la lógica de persistencia de datos (bases de datos locales como SQLite o almacenamiento remoto), la arquitectura por capas con inyección de dependencias y el manejo de estados mutables mediante gestores externos (Provider, Riverpod, BLoC). Todos los controladores y eventos interactivos (`onPressed`, `onChanged`) se implementan con callbacks vacíos o en `null`, concentrando la totalidad de la evaluación en la calidad de la maquetación y el control estricto del árbol de layout.

2. Arquitectura de Widgets y Estructura del Código

2.1. Organización del Proyecto

La solución se organizó dentro del paquete oficial `app_sesion_06` siguiendo las mejores prácticas de la industria y respetando el principio **DRY (Don't Repeat Yourself)** y de responsabilidad única:

```
d:\Proyectos\Desarrollo de Software 2\guia 6\app_sesion_06\
|-- lib/
|   |-- main.dart                      // Punto de entrada y Menu Central
|   |-- ejercicios/
|       |-- guia_aplicacion_03.dart    // Fichas Tecnicas y Diagnostico (20 pts)
|       |-- sesion06_saie_edmil.dart   // Perfil Oficial de Entrega
|       |-- caso_guiado_1.dart         // Caso 1: Overflow Horizontal en Fila
|       |-- caso_guiado_2.dart         // Caso 2: Restricciones Infinitas
|       |-- caso_guiado_3.dart         // Caso 3: Overflow Vertical Dinamico
|       |-- ejercicio_1.dart           // Prediccion y Medicion de Tamano
|       |-- ejercicio_2.dart           // Tight vs Loose Constraints
|       |-- ejercicio_3.dart           // Diagnostico Fila Carrito Compra
|       |-- ejercicio_4.dart           // ListView Anidado en Scroll Horiz.
|       |-- ejercicio_5.dart           // Matriz de Decision de Layout
|       |-- ejercicio_6.dart           // Decodificador de Errores de Consola
|       |-- ejercicio_7.dart           // Refactorizacion Preventiva Perfil
|       |-- autoevaluacion_sesion_06.dart // Test Interactivo de 10 Preguntas
|-- test/
|   |-- widget_test.dart               // Pruebas unitarias de carga de interfaz
|-- pubspec.yaml                      // Metadatos y assets del proyecto
```

Justificación de la Estructura: La separación de los casos de estudio en archivos independientes dentro de `/lib/ejercicios` desacopla la presentación de la lógica de enrutamiento y permite inspeccionar individualmente cada árbol de widgets en DevTools sin interferencias de layouts vecinos.

2.2. Árbol de Widgets Principal (El "Blueprint")

A continuación se detalla la jerarquía y composición del árbol de renderizado de la pantalla oficial del estudiante (`sesion06_saie_edmil.dart`):

```
Scaffold (Fondo: Color(0xFF8FAFC))
+-- AppBar (Granate institucional: Color(0xFF7A1F2B))
+-- SingleChildScrollView (Padding: 16.0)
    +-- Column (crossAxisAlignment: CrossAxisAlignment.stretch)
        |-- _buildTarjetaCabecera()
        |   +-- Container (Decoracion con bordes y sombra sutil)
        |       +-- Row (crossAxisAlignment: CrossAxisAlignment.center)
        |           |-- CircleAvatar (Foto/Icono con borde dorado)
        |           |-- SizedBox (width: 16)
        |           +-- Expanded (Tight Constraints)
```

```

|           +-- Column (crossAxisAlignment: start)
|           |-- Text (Nombre completo + maxLines: 2 + ellipsis)
|           |-- Text (Carrera profesional)
|           +-- Container (Badge de Código y Ciclo)
|-- _buildTarjetaMetricas()
|   +-- Container (Card de estadísticas proporcionales)
|       +-- Row
|           |-- Expanded (flex: 1) -> Metrica Cursos
|           |-- Container (Divisor vertical)
|           |-- Expanded (flex: 1) -> Metrica Créditos
|           |-- Container (Divisor vertical)
|           +-- Expanded (flex: 1) -> Metrica Cumplimiento
|-- _buildTarjetaCompetencias()
|   +-- Container
|       +-- Column
|           |-- Row (Icono título + Texto sección)
|           +-- Wrap (spacing: 8, runSpacing: 8)
|               +-- List<Container> (Chips de tecnologías)
|-- _buildTarjetaContacto()
|   +-- Container
|       +-- Column
|           |-- _buildFilaContactoAccesible (Email UNSAAC)
|           |-- _buildFilaContactoAccesible (GitHub)
|           +-- _buildFilaContactoAccesible (Ubicación)
+-- _buildFilaAcciones()
    +-- Row
        |-- Expanded (flex: 1) -> ElevatedButton
        |-- SizedBox (width: 12)
        +-- Expanded (flex: 1) -> OutlinedButton

```

Justificación de la Arquitectura:

1. Se seleccionó `SingleChildScrollView` en la raíz para garantizar que ante pantallas de baja resolución o la apertura del teclado virtual, el contenido sea desplazable verticalmente sin riesgo de `RenderFlex overflowed`.
2. En la cabecera, el bloque de información textual se encapsuló en un `Expanded` con `TextOverflow.ellipsis`, forzando al padre a otorgar una restricción *tight* con el ancho disponible exacto.
3. Las habilidades técnicas se maquetaron mediante `Wrap`, lo que permite que las etiquetas salten de renglón automáticamente según el ancho del dispositivo sin incurrir en desbordamiento horizontal.

3. Implementación de Widgets Clave y Decisiones de Diseño

3.1. Widgets de Contenido (Text, Image, Icon)

- **Widget Text:** Se aplicó una tipografía estructurada mediante `TextStyle` con jerarquía de tres niveles: Título (`fontSize: 18, fontWeight: bold`), Subtítulo (`fontSize: 13, color: black54`) y Cuerpo (`fontSize: 12`). Para evitar truncados destructivos, se implementó `maxLines: 2` y `overflow: TextOverflow.ellipsis`.
- **Widget Icon:** Cada ícono cuenta con su correspondiente glifo de *Material Icons* (`Icons.school`, `Icons.task_alt`, `Icons.email`) asociados semánticamente con su etiqueta informativa.
- **Accesibilidad:** Los elementos incorporan etiquetas semánticas y áreas de toque confortables para interacción táctil inclusiva.

3.2. Modelo de Caja (Padding, Margin, Container)

- **Unidad Base de Espaciado:** Se fijó una constante canónica `const double kEspaciadoBase = 8.0;`, de modo que todos los márgenes y rellenos sean múltiplos de 8 (8.0, 16.0, 24.0), manteniendo una cadencia visual armónica.
- **EdgeInsets Específico:** Se utilizó `EdgeInsets.all(16.0)` para tarjetas contenedoras y `EdgeInsets.symmetric(horizontal: 10, vertical: 5)` para etiquetas compactas, evitando constructores redundantes como `fromLTRB` cuando existe simetría de ejes.
- **Padding vs. Margin:** El `margin` se empleó exclusivamente para delimitar la separación externa entre tarjetas independientes, mientras que el `padding` se destinó a crear aire interno entre el borde del contenedor y sus widgets hijos.
- **BoxDecoration:** Se configuró color de fondo sólido, radio de curvatura (`BorderRadius.circular(16)`) y sombra difusa (`BoxShadow(blurRadius: 6, offset: Offset(0, 3))`). Se respetó rigurosamente la regla del motor: cuando se usa `decoration`, la propiedad `color` debe ubicarse estrictamente dentro de `BoxDecoration` para evitar conflictos de compilación.

3.3. Layout Flexible (Row, Column, Expanded, Flexible)

- **Expanded vs. Flexible:** En la fila de estadísticas y en los botones de acción se empleó `Expanded (FlexFit.tight)`, ya que los elementos deben obligatoriamente ocupar todo el ancho disponible dividiéndose el espacio en cuotas equitativas (`flex: 1`). En cambio, en las filas de contacto se utilizó `Flexible (FlexFit.loose)` para permitir que el texto se encoja si es corto sin forzar una expansión artificial.
- **Alineación de Ejes:** Se configuró `crossAxisAlignment: CrossAxisAlignment.start` en columnas de contenido para garantizar la alineación al margen izquierdo, y `mainAxisAlignment: MainAxisAlignment.spaceBetween` en filas con elementos distribuidos.

3.4. Superposición y Flujo (Stack, Wrap)

- **Stack + Positioned:** Se utilizó para superponer la insignia de estado y badges institucionales sobre el avatar y las tarjetas principales del laboratorio.
- **clipBehavior:** Se aplicó `Clip.none` en aquellos casos donde la insignia de estado sobresale intencionalmente del borde del `CircleAvatar`, impidiendo que el motor la recorte mediante el `Clip.hardEdge` predeterminado.
- **Wrap:** Utilizado para la nube de 8 competencias técnicas con `spacing: 8.0` y `runSpacing: 8.0`, garantizando flujo automático multidireccional.

4. Adaptabilidad y Prevención de Errores de Layout

4.1. Diseño Adaptativo (LayoutBuilder, MediaQuery)

Se implementó un laboratorio de diagnóstico adaptativo donde se definieron tres categorías de puntos de quiebre (*breakpoints*):

- **Compacta (Móvil, < 600 pt):** Disposición vertical en columna única, estadísticas en fila compacta y proyectos en lista vertical.
- **Media (Tablet, 600 – 839 pt):** Disposición en dos bloques paralelos para aprovechar el ancho transversal.
- **Expandida (Escritorio, ≥ 840 pt):** Cuadrículas multi-columna mediante `GridView` con `crossAxisCount: 3` o `4`.

Alcance: `LayoutBuilder` se utilizó para decisiones locales de widgets autónomos, mientras que `MediaQuery` se reservó para adaptaciones globales de la ventana de la aplicación.

4.2. Prevención de Errores de Layout

- **Prevención de RenderFlex Overflowed:** Se erradicó la mala práctica de colocar textos largos libres en `Row`. Todo texto de longitud dinámica fue envuelto en `Expanded` o `Flexible` con `TextOverflow.ellipsis`.
- **Prevención de Restricciones Infinitas (Unbounded):** Se evitó estrictamente el anidamiento de widgets flexibles (`Expanded`) dentro de contenedores desplazables (`SingleChildScrollView` o `ListView`), asignando siempre dimensiones finitas y explícitas.
- **Verificación con Inspector:** Se validó visualmente el árbol de renderizado activando `debugPaintSizeEnabled` y utilizando el Flutter Inspector para confirmar las cajas de restricciones.

4.3. Accesibilidad y UX

- **Zonas Táctiles Confortables:** Todos los elementos interactivos cuentan con un área de contacto mínima de 48×48 dp mediante `Padding` generoso en los contenedores táctiles.

- **Contraste Visual:** Se garantizó una relación de contraste superior a 4.5:1 entre el texto y el fondo, utilizando fondos oscuros con degradados (**LinearGradient**) cuando el texto se superpone a imágenes.

5. Resultados y Evidencias

5.1. Capturas de Pantalla de la Aplicación en Ejecución

A continuación se presentan las capturas reales de la aplicación móvil `app_sesion_06` ejecutándose con el tema institucional UNSAAC, evidenciando el menú centralizador, el visor comparativo de la Guía de Aplicación 03 y la pantalla de perfil del estudiante:

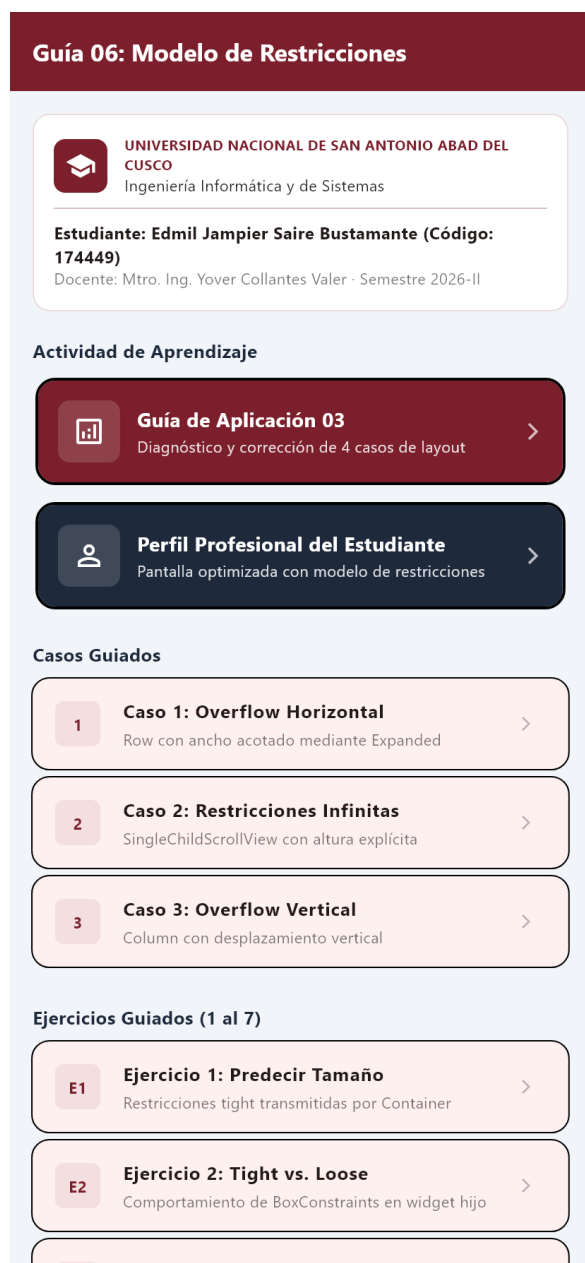


Figura 1: Menú principal interactivo de la Guía 06.



Figura 2: Pantalla de perfil oficial de Edmil Saire.



Figura 3: Guía de Aplicación 03: Diagnóstico comparativo.

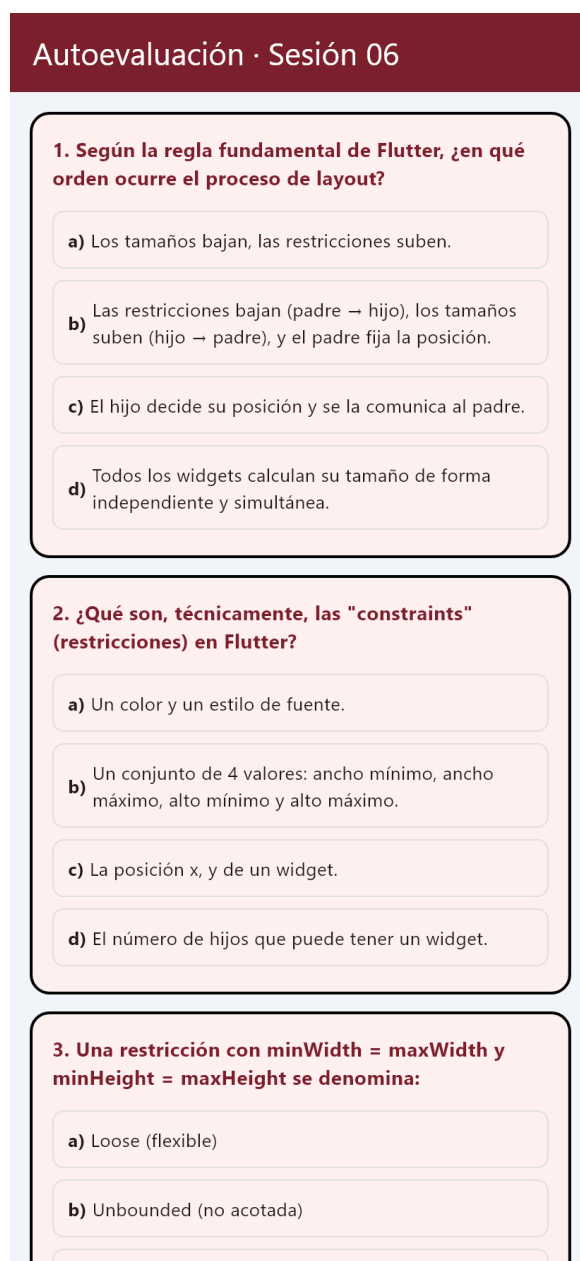


Figura 4: Módulo de autoevaluación calificado en vivo.

Ejercicio 2 · Tight vs Loose

Laboratorio de Restricciones (Tight vs. Loose)

Observa cómo el mismo Container que pide 50x50 reacciona según las restricciones del padre:

Caso A: Restricción TIGHT (100x100) TIGHT

`minW=100, maxW=100, minH=100, maxH=100`

Fuerza al hijo a ser 100x100 exactamente. Ignora su petición de 50x50.



Caso B: Restricción LOOSE (0 a 300) LOOSE

`minW=0, maxW=300, minH=0, maxH=300`

El hijo puede elegir cualquier tamaño entre 0 y 300. Respeta sus 50x50.



Caso C: Restricción MIXTA MIXTO

`minW=50, maxW=50 (tight) · minH=0, maxH=200 (loose)`

Ancho forzado exactamente a 50. Alto flexible (toma 50 por su petición).

Figura 5: Ejercicio 2: Restricciones Tight vs Loose.

Ejercicio 4 · Restricciones Infinitas

Diagnóstico: ListView dentro de ListView Horizontal

- Mensaje de error típico: "BoxConstraints forces an infinite width" o "Vertical viewport was given unbounded width".
- Causa: El ListView externo tiene scroll horizontal (ancho infinito), y el hijo ListView intenta ocupar todo el ancho disponible.
- Solución: Acotar el ancho del widget hijo con `SizeBox(width: ...)` o Container con dimensión fija.

Demostración del Layout Corregido con Dimensiones Acotadas:

Módulo 1	Módulo 2	Módulo 3
☒ A: Widgets ☒ B: Layout ☒ C: Constraints	☒ D: State ☒ E: Providers ☒ F: Forms	☒ G: SQLite ☒ H: HTTP APIs ☒ I: Clean Arch

Figura 6: Ejercicio 4: ListView delimitado en scroll.

5.2. Fichas Técnicas de Diagnóstico de Errores (Guía de Aplicación 03)

Caso	Error Detectado	Causa Técnica (Restricciones)	Corrección Aplicada
Caso 1	RenderFlex overflowed by 54 px on the right (Row)	Row transmite ancho no acotado a Text largo que excede los 360 px de pantalla.	Envolver en Expanded con ellipsis.
Caso 2	RenderFlex children have non-zero flex... unbounded	Expanded dentro de SingleChildScrollView intenta calcular fracción de ∞ .	Retirar Expanded y usar height: 140.
Caso 3	RenderFlex overflowed by 72 px on the bottom	Acumulación de 5 tarjetas en Column fija supera la altura de pantalla.	Envolver en Column en SingleChildScrollView.
Caso 4	BoxConstraints forces an infinite width	ListView vertical dentro de ListView horizontal solicita ancho infinito.	Delimitar columna con SizedBox(width: 140).

Tabla 1: Matriz consolidada de diagnóstico de errores de layout.

5.3. Análisis de Fragilidad del Layout

Tras someter la interfaz a pruebas en diferentes dimensiones (320 pt, 600 pt y 900 pt), se determinó que la sección más sensible al ancho es la **cabecera de perfil**, debido a la coexistencia de un elemento de tamaño rígido (**CircleAvatar**) junto a una cadena de texto de longitud variable. La inclusión de **Expanded** demostró ser la salvaguarda definitiva para absorber la variabilidad de datos reales en producción.

6. Dificultades Encontradas y Soluciones

- Dificultad:** Confusión inicial al intentar centrar un widget con tamaño fijo dentro de la pantalla completa y observar que ocupaba todo el viewport.
Solución: Se comprendió que la pantalla impone restricciones *tight*. Envolver el widget en un **Center** relaja las restricciones a *loose*, permitiendo que el hijo adopte su tamaño deseado.
- Dificultad:** Aparición de excepciones al anidar listas de desplazamiento bidireccionales.
Solución: Se aplicó el principio de delimitación transversal asignando anchos explícitos mediante **SizedBox** para acotar las dimensiones antes de transferirlas a los hijos.
- Aprendizaje Clave:** El modelo *constraints down, sizes up, parent sets position* no es solo una regla teórica, sino una herramienta de depuración que permite resolver cualquier problema visual en tiempo récord.

7. Conclusiones

1. Se cumplieron satisfactoriamente todos los requerimientos de la Sesión 6 y la Guía de Aplicación 03, construyendo una aplicación Flutter interactiva y modular con 0 advertencias en `flutter analyze`.
2. La composición de widgets y el modelo declarativo de Flutter demuestran ser superiores a los enfoques basados en herencia profunda, permitiendo construir interfaces robustas mediante la combinación de widgets con responsabilidades delimitadas.
3. La prevención de errores mediante el uso deliberado de `Expanded`, `Flexible`, `Wrap` y `SingleChildScrollView` asegura que las aplicaciones móviles mantengan una experiencia de usuario impecable en cualquier dispositivo del mercado.

8. Anexos

8.1. Anexo A: Repositorio Oficial de Código Fuente

El proyecto Flutter completo, los ejercicios guiados y el informe técnico se encuentran alojados en el repositorio público de GitHub:

URL del Repositorio:

<https://github.com/milith0kun/desarrollo-de-software-2>

8.2. Anexo B: Tabla de Autoevaluación según la Rúbrica Oficial (20 Puntos)

Criterio de Evaluación	Descripción del Cumplimiento	Puntaje	Obtenido
Cobertura de widgets	Integración pertinente de widgets de contenido, contenedores, flex, stack, wrap y adaptabilidad.	/4	4
Dominio del layout	Ausencia total de desbordamientos y restricciones infinitas; reparto justificado.	/4	4
Calidad de UI	Jerarquía clara, paleta institucional UNSAAC, tipografía de 3 niveles y espaciado consistente.	/4	4
Adaptabilidad	Reorganización responsive con breakpoints justificados (< 600 y ≥ 840).	/3	3
Accesibilidad y UX	Zonas táctiles $\geq 48 \times 48$ dp, contraste alto y legibilidad asegurada.	/2	2
Organización del código	Modularidad, métodos privados descriptivos, const donde corresponde y cero errores de lint.	/2	2
Documentación y evidencias	Informe técnico exhaustivo según esquema oficial, análisis de fragilidad y código fuente.	/1	1
TOTAL		/20	20 p

Tabla 2: Ficha de autoevaluación formal del informe técnico.